Programme	:	**B. Tech-ECM**	Semester	:	**Winter 23-24**
Course	:	**Artificial Intelligence and Machine Learning**	Code	:	**BECE309L**
Faculty	:	**Dr. Lekshmi K.**	Slot	:	**B2-TB2**

Travelling Salesperson Problem: Hill Climbing Algorithm with Interactive Visualization for Optimal Route Planning

Branch: B. Tech in Electronics and Computers ECM

School: Sense

Team members	
Name	***Reg num.***
Aditi Srivastava	***21BEC1214***
Tanisha Srivastava	***21BLC1664***
Rudraksh Chourey	***21BLC1098***
Aditya Shekhawat	***21BEC1232***
Rudransh Soni	***21BEC1264***

Submitted To: Dr.Lekshmi K.

Pg-14

Hill Climbing Algorithm

Hill Climbing is a heuristic search algorithm used for mathematical optimization problems. It tries to find the local optimum (maximum or minimum) of a given function. The algorithm starts with an arbitrary solution to a problem and then iteratively makes small changes to it, each time improving the solution. **It keeps moving towards the direction that gives the best improvement, akin to climbing up a hill to reach the peak. Hill Climbing terminates when it reaches a peak where no immediate neighbour has a higher value (for maximization) or a valley where no immediate neighbour has a lower value (for minimization).**

Initialization: Start with an initial solution.

Evaluation: Evaluate the current solution.

Neighbour Generation: Generate neighbouring solutions by making small changes to the current solution.

Selection: Select the best neighbouring solution based on the evaluation function.

Termination: Check if the stopping criteria are met. If not repeat the step.

Hill Climbing Algorithm is broadly employed in optimization scenarios across diverse fields, offering a flexible approach to finding optimal solutions within complex search spaces. Its generalized applications encompass:

Optimization Challenges: Addressing optimization challenges in scheduling, resource allocation, and parameter tuning without exhaustive exploration of the entire solution space.

Heuristic Search Strategies: Supporting heuristic search strategies to efficiently navigate vast search spaces, enabling the identification of satisfactory solutions without exhaustive examination.

Local Search Problem Solving: Excelling in local search problems by iteratively refining solutions until acceptable outcomes are achieved, prioritizing local optimality over global perfection.

Game Strategy Formulation: Informing game-playing AI by evaluating potential moves and selecting the most advantageous ones, enhancing strategic decision-making in various gaming scenarios.

Efficient Routing and Navigation: Facilitating efficient routing and navigation systems by determining optimal paths considering factors like distance, traffic, and obstacles.

Thus, Hill Climbing Algorithm serves as a versatile optimization tool applicable to a multitude of scenarios across various domains, facilitating the discovery of optimal or near-optimal solutions efficiently and effectively.

The Hill Climbing Algorithm employs a greedy search approach, iteratively selecting the locally optimal move at each step to optimize solutions. It focuses on improving the current solution by making incremental adjustments, prioritizing immediate gains without extensive consideration of future consequences. This strategy entails exploring neighboring solutions and selecting the one with the highest improvement, akin to climbing towards a local peak. However, this approach may lead to

suboptimal solutions if it gets stuck at a local optimum without exploring other possibilities, illustrating its inherent trade-off between exploration and exploitation in search spaces.

Traveling Salesman Problem

The Traveling Salesman Problem (TSP) has long been a benchmark challenge in optimization, requiring the determination of the shortest possible route for a salesman to visit a set of cities exactly once and return to the starting city. This document offers a comprehensive exploration of TSP, with a focus on heuristic and metaheuristic algorithms devised to tackle its inherent complexity. In particular, it scrutinizes the application of Hill Climbing, a foundational heuristic approach, in optimizing TSP solutions. While Hill Climbing presents simplicity, its susceptibility to converging on local optima poses challenges. Nevertheless, through an in-depth analysis of its mechanics and the intricacies of TSP, this document underscores the pivotal role of Hill Climbing in addressing this renowned optimization challenge.

Heuristic and Metaheuristic Approaches:

Heuristic methods offer practical problem-solving strategies that prioritize efficiency over optimality, often relying on intuition and past experience. In contrast, metaheuristic algorithms provide higher-level frameworks for exploring solution spaces, employing techniques like randomization and adaptive search. These approaches are particularly relevant for complex optimization problems like TSP, where finding exact solutions is computationally intractable due to the problem's combinatorial nature and large solution space.

Understanding Hill Climbing: A Fundamental Heuristic:

Hill Climbing stands as a foundational heuristic approach, aiming to iteratively improve a solution by making incremental changes. At each iteration, the algorithm evaluates neighboring solutions and selects the one with the highest improvement, akin to climbing a hill to reach a local peak. However, Hill Climbing operates under a local search strategy, focusing solely on immediate gains without considering long-term consequences extensively. This characteristic makes it susceptible to converging on local optima and potentially overlooking globally optimal solutions.

Application of Hill Climbing in Tackling TSP:

Hill Climbing finds application in addressing the optimization challenges posed by TSP. In this context, cities represent states, and routes represent solutions. The algorithm iteratively explores neighboring solutions by making small adjustments to the current route, aiming to find shorter paths and improve upon the current solution iteratively. Despite its simplicity, Hill Climbing offers a straightforward approach to navigating the complex solution space of TSP and identifying near-optimal routes.

Limitations and Challenges of Hill Climbing in TSP Optimization:

Despite its utility, Hill Climbing has inherent limitations when applied to TSP optimization. Its local search strategy may lead to premature convergence on local optima, resulting in suboptimal routes. Additionally, the vast solution space of TSP presents a significant challenge, as exploring all possible routes becomes impractical for large problem instances. These limitations underscore the need for complementary or alternative approaches to effectively tackle TSP optimization.

Beyond Hill Climbing: Advanced Algorithms for TSP:

Advanced optimization algorithms like Simulated Annealing and Genetic Algorithms offer alternative strategies for TSP optimization. Simulated Annealing introduces probabilistic transitions between solutions to escape local optima, while Genetic Algorithms use evolutionary principles like mutation and crossover to explore the solution space more comprehensively. These algorithms provide promising avenues for overcoming the limitations of Hill Climbing and achieving better optimization performance in TSP.

Conclusion: The Role of Hill Climbing in Tackling TSP Optimization Challenges:

Hill Climbing plays a crucial role as a foundational heuristic approach in addressing the optimization challenges posed by TSP. Despite its limitations, it offers a straightforward strategy for navigating the solution space and identifying near-optimal routes. However, the complexity of TSP necessitates consideration of complementary or alternative approaches, such as advanced optimization algorithms, to achieve more robust and effective solutions. Ongoing research and development efforts aim to further advance the state-of-the-art in TSP optimization, reflecting the continuous quest for innovative solutions to complex optimization problems.

Algorithm Used

The major algorithm used in the code for solving the Traveling Salesman Problem (TSP) is a Hill Climbing Algorithm using greedy approximation approach.

Here's how the algorithm works in the context of the provided code:

Initialization:

The code first initializes a directed graph G where each city is represented as a node. It also initializes an empty route list to store the approximate solution.

Hill Climbing Algorithm using specifically Nearest Neighbour/ Greedy approach:

The **approx_tsp** method implements the Hill Climbing Algorithm. It starts from a given starting city and iteratively selects the nearest unvisited city to the current city and adds it to the route. This process continues until all cities are visited, forming a cycle.

Distance Calculation:

The algorithm calculates the distance between each pair of cities using the Haversine formula, which calculates the great-circle distance between two points on the Earth's surface given their longitudes and latitudes.

Rendering the TSP Graph:

After computing the approximate solution, the **render_tsp_graph** method visualizes the route on a graph. It plots the cities as nodes and the edges represent the connections between cities in the order they are visited in the route. The starting city is highlighted with a different color and shape to distinguish it from other cities.

Rendering the Distance Graph:

Additionally, the code provides the option to visualize the distances between city pairs in a separate graph. This graph displays all cities as nodes, and the edges represent the distances between pairs of cities.

User Interaction:

The code utilizes the **Tkinter library to create a graphical user interface (GUI)** where users can input cities, select a starting city, and visualize the solution and distance graph. Overall, the Hill Climbing algorithm provides a simple and efficient approach to find an approximate solution to the TSP, making it suitable for practical use, especially for large datasets where exact solutions are computationally expensive.

Heuristic Function

The heuristic function used in the code is implemented within the **approx_tsp** method, specifically in the line:

nearest_city = min(unvisited_cities, key=lambda city: G[current_city][city]['weight'])

This line employs the **min()** function to find the nearest unvisited city **(nearest_city)** to the current city **(current_city)** based on the weight of the edges connecting them. The key parameter of the **min()** function specifies a lambda function that extracts the weight of the edge between the current city and each unvisited city. The **lambda city: G[current_city][city]['weight']** part of the code retrieves the weight (distance) of the edge between the current city and each unvisited city from the graph G, and min() selects the city with the minimum weight, i.e., the nearest city.

This heuristic function guides the Nearest Neighbor algorithm's decision-making process by favoring the selection of nearby cities at each step, aiming to construct a tour with a shorter overall distance.

Thus, the distance between the cities are taken as Heuristic Values for the cities and to apply Hill Climbing Algorithm.

Libraries Used

The code utilizes several libraries to implement functionality for solving the Traveling Salesman Problem (TSP) and creating a graphical user interface (GUI) for user interaction:

- **tkinter:** This is the standard GUI (Graphical User Interface) toolkit in Python. It is used for creating windows, buttons, labels, entry fields, and other GUI elements. In this code, tkinter is used to create the main application window (Tk()), labels (Label), buttons (Button), entry fields (Entry), and option menus (OptionMenu).
- **geopy:** Geopy is a Python library used for geocoding and distance calculations. In this code, it is used to calculate the geographical distance between pairs of cities using the geodesic function. It retrieves the latitude and longitude coordinates of cities using the Nominatim geocoder.
- **networkx:** NetworkX is a Python library for creating, analyzing, and visualizing complex networks (graphs). Here, it is used to represent the cities and their connections (edges) as a graph. NetworkX provides functions for adding nodes and edges to the graph, as well as for performing graph algorithms like finding the shortest path.
- **matplotlib:** Matplotlib is a plotting library for Python. It is used to create visualizations, such as line plots, scatter plots, histograms, and bar charts. In this code, Matplotlib is used to create and display graphs representing the TSP solution and the distances between city pairs. The FigureCanvasTkAgg class from matplotlib.backends.backend_tkagg is used to embed Matplotlib figures into the tkinter GUI.

Overall, these libraries provide the necessary tools and functionality to implement the TSP solver and create an interactive GUI for users to input cities, visualize solutions, and interact with the application.

Overview of the Code

The provided code is a Python implementation of a Traveling Salesman Problem (TSP) solver with a graphical user interface (GUI) using tkinter for interaction and visualization. Here's a breakdown of its functionality:

- **Initialization:** The TSPSolver class initializes the GUI window (Tk()), sets its title and geometry, and loads a background image. It also initializes variables to store city data, distances between cities, and GUI elements such as labels, buttons, and entry fields.
- **GUI Elements:** The GUI consists of an entry field for users to input city names, buttons to add cities and proceed to the next step, and a frame to display graphs. Fonts and event bindings for font adjustment are also defined.
- **Event Handlers:** Methods like add_city, next_page, and confirm_start_city handle user interactions. add_city adds cities entered by the user to a list. next_page verifies that at least two cities are entered before proceeding to the next step. confirm_start_city triggers TSP solving and graph rendering when the starting city is selected.
- **TSP Solver:** The solve_tsp method constructs a directed graph using NetworkX, calculates distances between cities using geopy, and solves the TSP using a nearest neighbor heuristic in approx_tsp. This algorithm selects the nearest unvisited city at each step to construct a tour.
- **Graph Rendering:** The render_tsp_graph and render_distance_graph methods use NetworkX and Matplotlib to visualize the TSP solution and distances between city pairs, respectively.
- **Graph Toggling:** The toggle_graph method switches between displaying the TSP solution and distance graphs based on a toggle button's state.
- **Main Execution:** The __main__ block initializes and runs the TSPSolver class.

In summary, the code combines algorithms for solving the TSP, geocoding, and graphical visualization to provide an interactive tool for users to input cities, visualize TSP solutions, and explore city distances.

Test Case:

The test case involves simulating user interaction to input cities and select a starting city using the provided GUI interface. Here's a breakdown of the test case and its solution:

Initialization: The test case initializes an instance of the TSPSolver class, creating the GUI window and setting up necessary components for city input and selection.

City Input: The test case simulates user input by manually adding the specified cities ("Delhi", "Panaji", "Bhopal", "Lucknow", and "Hyderabad") to the city entry field using the city_entry.insert() method and then adding them to the list of cities using the add_city() method.

Proceed to Next Step: After adding all cities, the test case proceeds to the next step in the GUI interface using the next_page() method. This step confirms that at least two cities have been added before advancing.

Starting City Selection: The test case selects "Delhi" as the starting city by setting the value of the start_city_var variable to "Delhi" and then confirming the selection using the confirm_start_city() method. This triggers the TSP solving process.

TSP Solution: The code calculates the TSP solution using the Nearest Neighbor algorithm implemented in the approx_tsp() method. It selects the nearest unvisited city at each step to construct a tour, aiming to minimize the overall tour length.

Graph Visualization: After solving the TSP, the code visualizes the solution on a graph using NetworkX and Matplotlib. It displays the cities and the connections between them, with the starting city highlighted, providing a graphical representation of the optimal tour.

In summary, the test case validates the functionality of the TSP solver by providing a set of cities, selecting a starting city, and visualizing the solution on a graph through the GUI interface. It demonstrates how the code handles user input, solves the TSP, and presents the solution for analysis and verification.